

BBO on trade (TBBO)

BBO on trade (TBBO) provides every trade event alongside the BBO immediately *before* the effect of each trade. This is a subset of **MBP-1**.

Fields (tbbo)

Field	Type	Description
ts_recv	uint64_t	The capture-server-received timestamp expressed as the number of nanoseconds since the UNIX epoch. See ts_recv .
ts_event	uint64_t	The matching-engine-received timestamp expressed as the number of nanoseconds since the UNIX epoch. See ts_event .
rtype	uint8_t	A sentinel value indicating the record type. Always 1 in the TBBO schema. See Rtype .
publisher_id	uint16_t	The publisher ID assigned by Databento, which denotes the dataset and venue. See Publishers .
instrument_id	uint32_t	The numeric instrument ID. See Instrument identifiers .
action	char	The event action. Always Trade in the TBBO schema. See Action .
side	char	The side that initiates the trade. Can be Ask for a sell aggressor in a trade, Bid for a buy aggressor in a trade, or None where no side is specified. See Side .
depth	uint8_t	The book level where the update event occurred.
price	int64_t	The order price where every 1 unit corresponds to 1e-9, i.e. 1/1,000,000,000 or 0.000000001. See Prices .
size	uint32_t	The order quantity.
flags	uint8_t	A bit field indicating event end, message characteristics, and data quality. See Flags .
ts_in_delta	int32_t	The matching-engine-sending timestamp expressed as the number of nanoseconds before ts_recv . See ts_in_delta .
sequence	uint32_t	The message sequence number assigned at the venue.
bid_px_00	int64_t	The bid price at the top level where every 1 unit corresponds to 1e-9, i.e. 1/1,000,000,000 or 0.000000001. See Prices .
ask_px_00	int64_t	The ask price at the top level where every 1 unit corresponds to 1e-9, i.e. 1/1,000,000,000 or 0.000000001. See Prices .
bid_sz_00	uint32_t	The bid size at the top level.
ask_sz_00	uint32_t	The ask size at the top level.
bid_ct_00	uint32_t	The bid order count at the top level.
ask_ct_00	uint32_t	The ask order count at the top level.

Implementation differences between clients and encodings

Bid and ask depth messages (fields starting with **bid_** and **ask_**) are structured differently in the **C++ and Rust clients**, the **Python record interface**, and **JSON data**. Instead of using the **_N** suffix, they're stored in an array of structures named **levels**, with the top-of-book at index **0**.

For example, in C++, **levels[5].bid_px** corresponds to **bid_px_05** in the Python DataFrame API and CSV format.